



Universidad de Talca

Facultad de Ingeniería

Informe de Tarea 4

Generación Procedimental de Contenido: el caso de Soul Knight

Asignatura: Inteligencia Artificial

Profesor: Nicolas Alejandro Romero Emparanza

Estudiante: Diego Cabrera y Benjamin Llauca

Carrera: Ingeniería en Desarrollo de Videojuegos y Realidad Virtual

Talca, 2026

Abstract

1. Introducción

La generación procedimental de contenido (*Procedural Content Generation*, PCG) se ha convertido en una herramienta central dentro de la industria de los videojuegos, particularmente en géneros donde la rejugabilidad depende de que cada sesión presente un escenario distinto. Esta técnica permite reducir el costo de producción asociado al diseño manual de niveles y, al mismo tiempo, sostener el interés del jugador mediante la variabilidad del contenido generado. Junto con la generación de niveles, los videojuegos que dependen de combate en tiempo real requieren además sistemas de inteligencia artificial capaces de mover a los enemigos de forma creíble dentro de esos escenarios generados dinámicamente, lo que introduce el problema de la búsqueda de caminos (*pathfinding*). El presente informe aborda ambos problemas, que sería la generación procedimental de mazmorras y búsqueda de caminos para el comportamiento enemigo, tomando como caso de estudio el videojuego *Soul Knight*.

1.1. Definición del Proyecto

Soul Knight es un videojuego de tipo *roguelike* de disparos vista cenital (*top-down shooter*), desarrollado por el estudio independiente ChillyRoom y lanzado originalmente en 2017 para dispositivos móviles. El juego sitúa al jugador en una mazmorra dividida en pisos compuestos por múltiples salas conectadas entre sí, cada partida genera una distribución distinta de salas, enemigos, obstáculos y recompensas, lo que constituye su mecánica central de rejugabilidad. A nivel de diseño, *Soul Knight* comparte una estructura de combate y generación de niveles emparentada con la de otros *bullet-hell roguelites* de cámara cenital, género del cual además declara explícitamente su inspiración. El jugador debe recorrer las salas generadas, enfrentando distintos tipos de enemigos a melé, a distancia y jefes, mientras recolecta armas y mejoras antes de avanzar al siguiente nivel.

Para este informe se seleccionaron dos enfoques de trabajo:

1. **Generación procedimental de la estructura de la mazmorra**, mediante el algoritmo de *Binary Space Partitioning* (BSP). El algoritmo BSP subdivide recursivamente un espacio rectangular en particiones cada vez más pequeñas hasta obtener una jerarquía de nodos hoja, que luego se transforman en salas y un proceso adicio-

nal se encarga de conectar dichas salas mediante pasillos. Esta técnica es coherente con la estructura del juego elegido, ya que *Soul Knight* organiza sus niveles como un conjunto de salas rectangulares discretas conectadas por corredores.

2. **Decoración procedural del contenido interior de cada sala**, complementaria al BSP, que se encarga de poblar las salas generadas con enemigos, obstáculos destructibles, trampas y recompensas, replicando la variabilidad de contenido que presenta el juego original dentro de cada sala individual.

En paralelo a la generación de la mazmorra, se busca dotar a los enemigos de comportamientos de búsqueda de caminos diferenciados según su rol dentro del combate, utilizando para ello cuatro algoritmos de búsqueda con propiedades distintas:

- **Depth First Search (DFS)**: además de su uso clásico en la exploración de grafos, se empleará como mecanismo de conexión de pasillos entre las salas generadas por el BSP, recorriendo el árbol de partición en profundidad para garantizar que todas las salas queden enlazadas en una estructura conexa y sin ciclos redundantes.
- **Breadth First Search (BFS)**: se aplicará al comportamiento de los enemigos melé de bajo nivel (equivalentes a los goblins o esqueletos del juego original), cuyo costo de desplazamiento por celda es uniforme. Al no requerir ponderaciones, BFS entrega el camino más corto en este tipo de grilla con un costo computacional menor al de alternativas más sofisticadas, lo que resulta apropiado para enemigos numerosos que deben perseguir al jugador de forma simple y predecible.
- **A***: se utilizará para enemigos a distancia (arqueros o magos), que necesitan no solo perseguir al jugador sino encontrar posiciones de disparo óptimas evitando obstáculos del escenario. La heurística de A* permite obtener el camino óptimo dirigiendo la búsqueda hacia el objetivo, evitando la exploración exhaustiva que realizaría BFS sobre el mismo mapa.
- **Weighted A* (WA*)**: se reservará para jefes, donde la velocidad de respuesta en tiempo real es más relevante que la optimalidad estricta del camino. Al ponderar la heurística por encima de su valor admisible, WA* sacrifica algo de optimalidad a cambio de menor tiempo de cómputo, lo cual es deseable en encuentros donde el enemigo debe reaccionar de inmediato a los movimientos del jugador.

La relación entre las características de *Soul Knight* y los algoritmos seleccionados son coherentes, por lo que en dos ejes tenemos: por un lado, la estructura del juego en salas discretas conectadas hace que BSP sea una técnica natural para reproducir su distribución

espacial. Por otro, la diversidad de roles de combate de sus enemigos, desde unidades simples hasta jefes, el uso de un conjunto de algoritmos de búsqueda con distintos balances entre optimalidad y velocidad, es mejor que en lugar de un único algoritmo aplicado de manera uniforme a todos los enemigos del juego.

1.2. Entorno de Desarrollo

La implementación de los algoritmos seleccionados se realizará en `C++`, utilizando exclusivamente la biblioteca estándar del lenguaje (STL) para las estructuras de datos requeridas por los algoritmos de búsqueda (colas, pilas, colas de prioridad y estructuras tipo *heap* para A^* y WA^*). `C++` por su control explícito sobre la memoria y su rendimiento en operaciones repetitivas sobre grillas, características relevantes dado que los algoritmos de búsqueda de caminos deben ejecutarse en tiempo real para múltiples enemigos de forma simultánea. La visualización de los mapas generados se realizará mediante una representación matricial impresa en consola (modo texto), suficiente para evaluar la correctitud y las propiedades estructurales de los mapas generados sin depender de un motor gráfico externo. El proyecto incluirá un archivo `main.cpp`, un `Makefile` para su compilación y un archivo `ReadMe.md` con las instrucciones de ejecución.

1.3. Repositorio del Proyecto

El repositorio público del proyecto, actualmente en preparación, se encuentra disponible en:

<https://github.com/Condordie/Tarea4-IA>

En síntesis, el problema abordado en este informe combina dos objetivos de la generación procedimental aplicada a videojuegos: la construcción del espacio jugable y la inteligencia de los agentes que lo habitan. Aplicar BSP junto con un conjunto diferenciado de algoritmos de búsqueda permite abordar de forma coherente con la estructura y las mecánicas de *Soul Knight*, sentando además una base extensible para incorporar, en etapas posteriores de la Tarea 4, parámetros configurables y métricas de evaluación sobre los mapas y comportamientos generados.

2. Estado del Arte

La generación procedimental de mazmorras mediante *Binary Space Partitioning* ha sido documentada en trabajos recientes que combinan el algoritmo base con técnicas complementarias. Se propone una técnica que combina BSP con sistemas de Lindenmayer (L-Systems) para generar los pasillos que conectan las salas producidas por la partición del espacio, reemplazando los métodos de conexión más simples basados en retroceso (*backtracking*) por reglas gramaticales estocásticas que permiten controlar parámetros como el ancho de los corredores y los ángulos de giro permitidos. Por otra parte, documentan un caso de estudio práctico de un generador de mazmorras en Unity que utiliza BSP como algoritmo central, exponiendo parámetros ajustables de dimensión, densidad de salas y elementos decorativos. Los autores destacan que, para garantizar la conectividad confiable entre salas, los métodos de conexión basados directamente en el árbol BSP pueden ser inestables, por lo que aplican un método inspirado en búsqueda en profundidad (DFS) para la generación de los pasillos, reforzando la pertinencia de DFS como herramienta de conexión en este tipo de generadores y no únicamente como algoritmo de búsqueda en grafos.

Más allá de BSP, la literatura sobre generación de niveles para juegos de mazmorra contempla otras familias de técnicas que no fueron seleccionadas para este proyecto pero que resultan relevantes como contraste. Entre ellas se encuentran los autómatas celulares, adecuados para mapas de aspecto más orgánico y menos compartimentado que el de *Soul Knight*. El algoritmo de *Wave Function Collapse*, que construye el contenido a partir de restricciones locales entre piezas predefinidas y enfoques basados en aprendizaje automático, como las redes generativas adversariales (GAN), que aprenden patrones de niveles a partir de ejemplos diseñados por humanos en lugar de reglas explícitas. Estas alternativas comparten con BSP el objetivo de producir variabilidad y reducir el esfuerzo manual de diseño, pero exigen mayor costo de implementación o de datos de entrenamiento, lo que las hace menos apropiadas para el alcance de este informe. En cuanto a otros videojuegos que comparten características estructurales con *Soul Knight*, destaca particularmente *Enter the Gungeon*, *bullet-hell roguelite* de cámara cenital del cual *Soul Knight* declara abiertamente su inspiración, ambos títulos comparten la organización del nivel en salas discretas conectadas por pasillos y pobladas de enemigos y recompensas generados de forma distinta en cada partida, lo que confirma la pertinencia de técnicas de partición espacial como BSP para este subgénero de *roguelikes* de combate.

Respecto a los algoritmos de búsqueda de caminos para el comportamiento de los enemi-

gos, se realiza una revisión sistemática de algoritmos de búsqueda basados en inteligencia aplicados a videojuegos, cubriendo desde métodos no informados como BFS y DFS hasta variantes informadas y ponderadas de A^* , y discuten los criterios, tiempo de cómputo, optimalidad del camino y adaptabilidad a entornos dinámicos. Que determinan la elección de un algoritmo según el género y las restricciones de tiempo real del juego. En la misma línea, pero con un enfoque experimental más acotado, se estudia la optimización de A^* mediante una estructura de *heap* binario para gestionar la lista abierta del algoritmo en un videojuego de terror, reportando una reducción de aproximadamente un 49% en el tiempo de cómputo del algoritmo al sustituir una lista lineal por un *heap*, lo que se traduce en enemigos con persecución más rápida y reactiva. Este resultado es particularmente relevante para la justificación de WA^* en el presente proyecto, ya que de forma empírica el principio según el cual, en escenarios de combate en tiempo real, sacrificar algo de optimalidad o invertir en optimizaciones de implementación a cambio de mayor velocidad de respuesta puede mejorar la experiencia de juego percibida. Como técnicas de búsqueda adicionales que podrían haberse aplicado al mismo problema, pero que no fueron seleccionadas para este informe, cabe mencionar el algoritmo de Dijkstra equivalente a A^* sin heurística, útil cuando no se conoce un único objetivo fijo y *Jump Point Search*, una optimización de A^* para grillas uniformes que omite la evaluación de nodos intermedios redundantes en línea recta.

3. Metodología

4. Resultados

5. Discusión

6. Conclusiones

7. Referencias

Referencias